ASSIGNMENT – 2.1

NAME : G.RISHITHA

HT.NO : 2303A52149

BATCH : 35

Lab 2: Exploring Additional AI Coding Tools beyond Copilot – Gemini (Colab) and Cursor AI.

Task 1: Statistical Summary for Survey Data

Scenario: You are a data analyst intern working with survey responses stored as numerical lists.

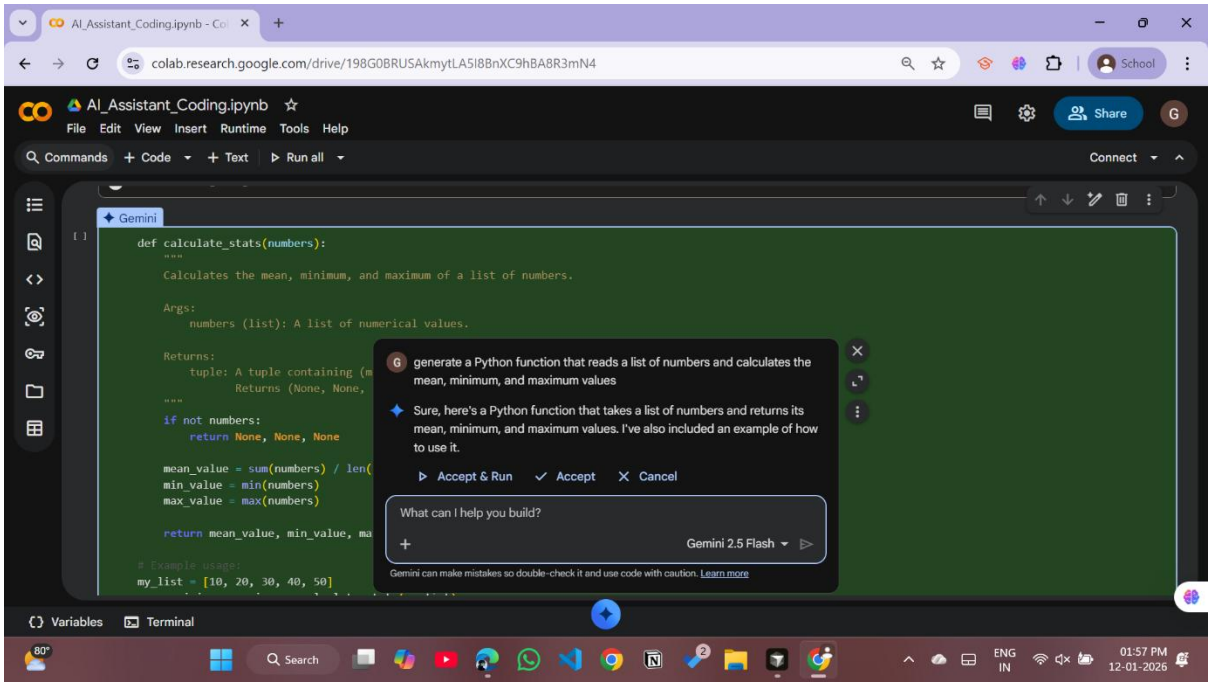
Task: Use Google Gemini in Colab to generate a Python function that reads a list of numbers and calculates the mean, minimum, and maximum values.

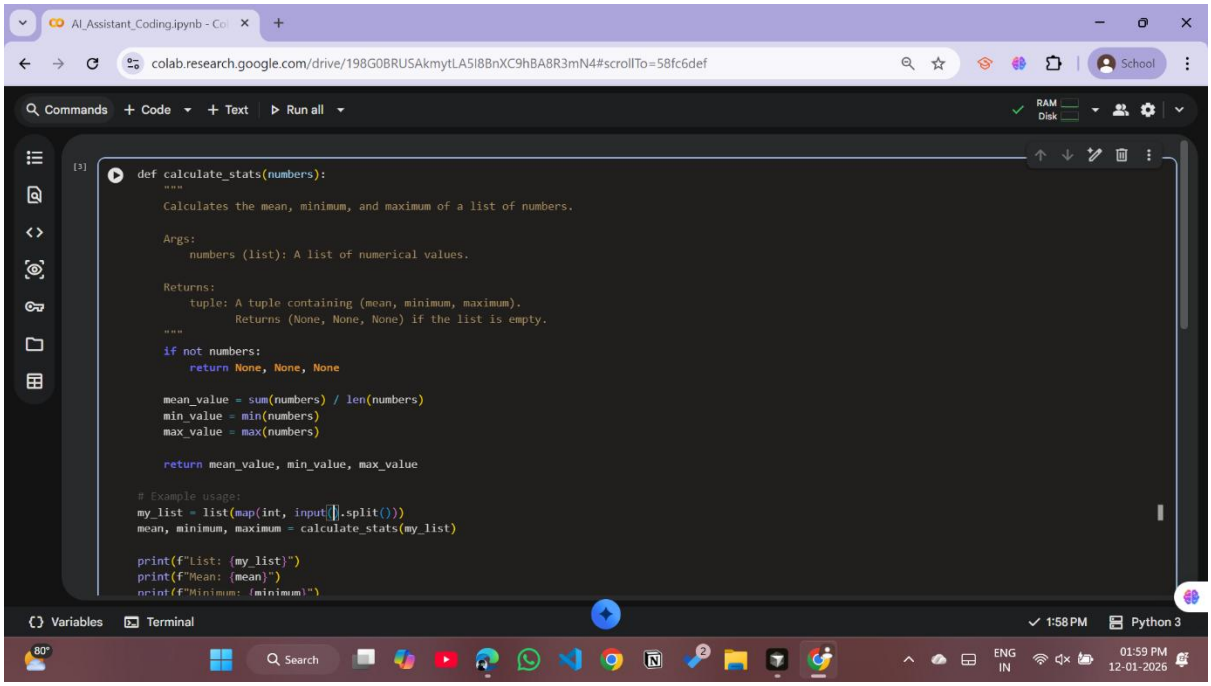
Expected Output:

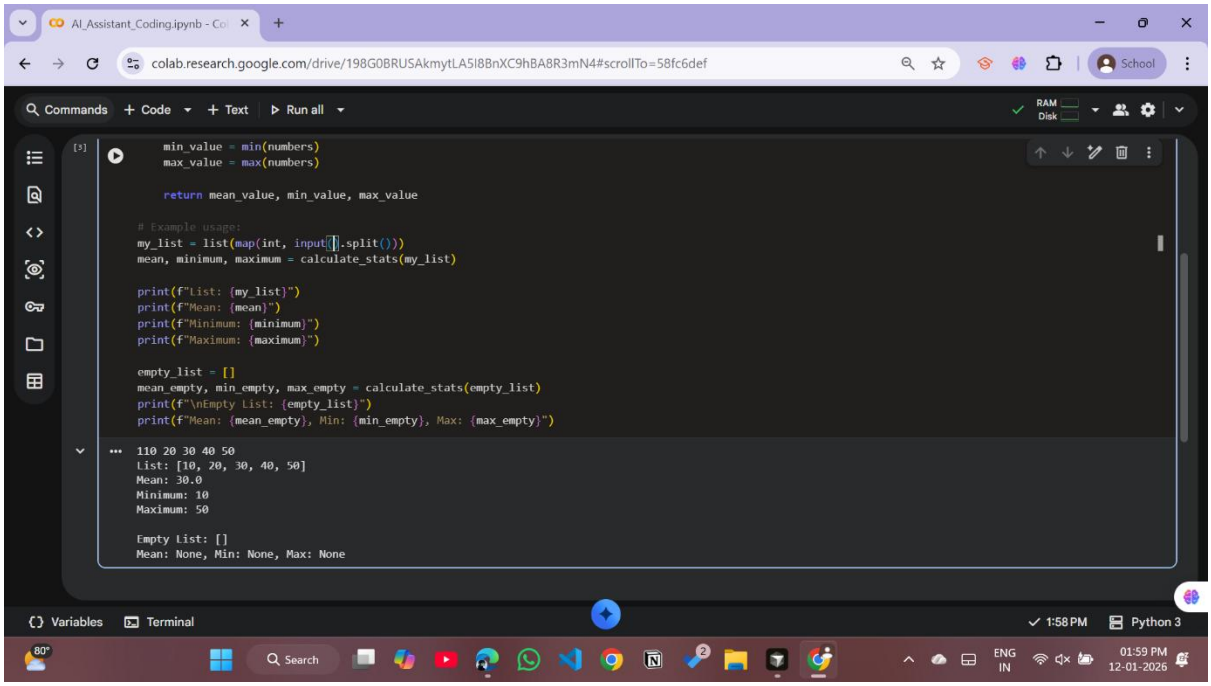
Correct Python function

Output shown in Colab

Screenshot of Gemini prompt and result


AI_Assistant_Coding.ipynb
colab.research.google.com/drive/198G0BRUSAkmytLA5I8BnXC9hBA8R3mN4
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
Connect
Share
Gemini
def calculate_stats(numbers):
"""
Calculates the mean, minimum, and maximum of a list of numbers.
Args:
numbers (list): A list of numerical values.
Returns:
tuple: A tuple containing (m
Returns (None, None,
"""
if not numbers:
return None, None, None
mean_value = sum(numbers) / len(
min_value = min(numbers)
max_value = max(numbers)
return mean_value, min_value, ma
# Example usage:
my_list = [10, 20, 30, 40, 50]
generate a Python function that reads a list of numbers and calculates the mean, minimum, and maximum values
Sure, here's a Python function that takes a list of numbers and returns its mean, minimum, and maximum values. I've also included an example of how to use it.
Accept & Run
Accept
Cancel
What can I help you build?
Gemini 2.5 Flash
Gemini can make mistakes so double-check it and use code with caution. Learn more
Variables
Terminal
Search
ENG IN
01:57 PM 12-01-2026


AI_Assistant_Coding.ipynb
colab.research.google.com/drive/198G0BRUSAkmytLA5I8BnXC9hBA8R3mN4#scrollTo=58fc6def
Commands + Code + Text Run all
RAM
Disk
[3]
def calculate_stats(numbers):
"""
Calculates the mean, minimum, and maximum of a list of numbers.
Args:
numbers (list): A list of numerical values.
Returns:
tuple: A tuple containing (mean, minimum, maximum).
Returns (None, None, None) if the list is empty.
"""
if not numbers:
return None, None, None
mean_value = sum(numbers) / len(numbers)
min_value = min(numbers)
max_value = max(numbers)
return mean_value, min_value, max_value
# Example usage:
my_list = list(map(int, input().split()))
mean, minimum, maximum = calculate_stats(my_list)
print(f"List: {my_list}")
print(f"Mean: {mean}")
print(f"Minimum: {minimum}")
Variables
Terminal
1:58 PM
Python 3
Search
ENG IN
01:59 PM 12-01-2026



Task 2: Armstrong Number – AI Comparison

Scenario: You are evaluating AI tools for numeric validation logic.

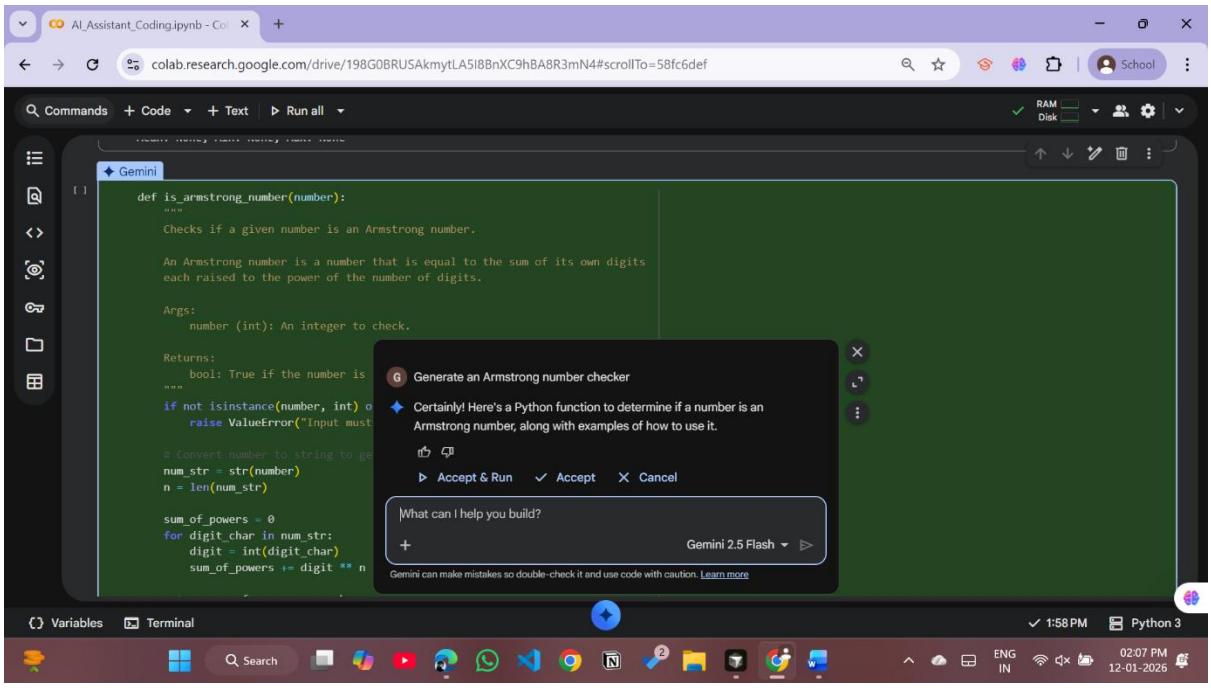
Task: Generate an Armstrong number checker using Gemini and GitHub Copilot. Compare their outputs, logic style, and clarity.

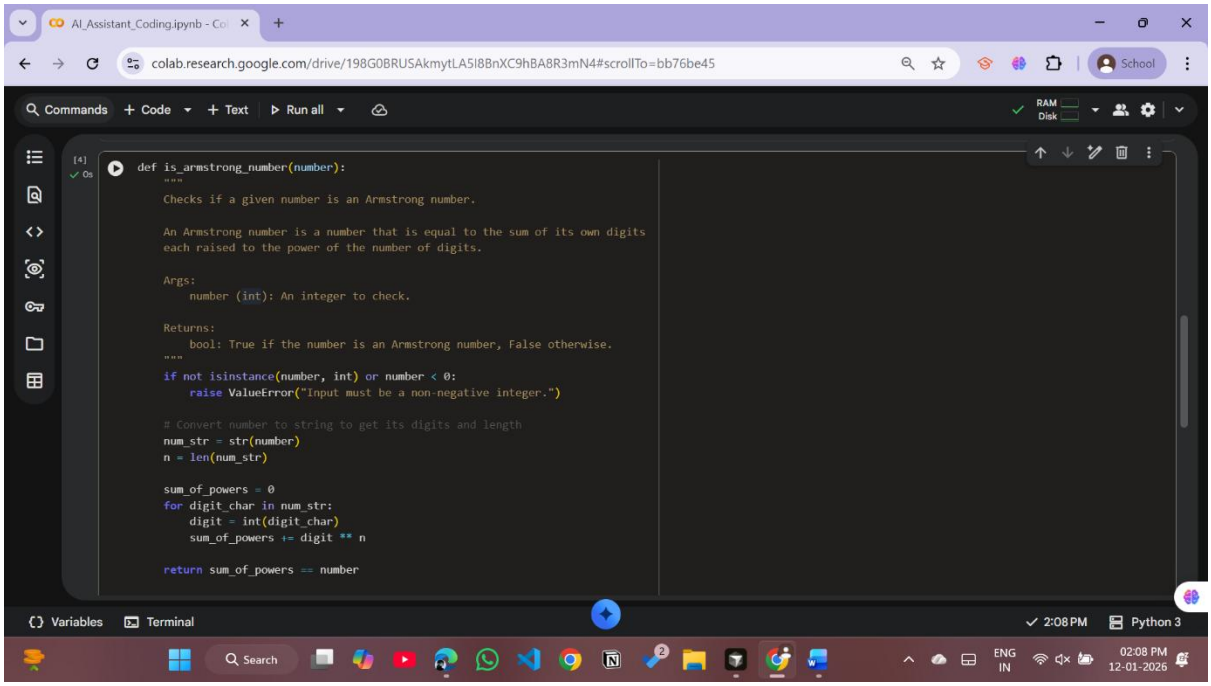
Expected Output:

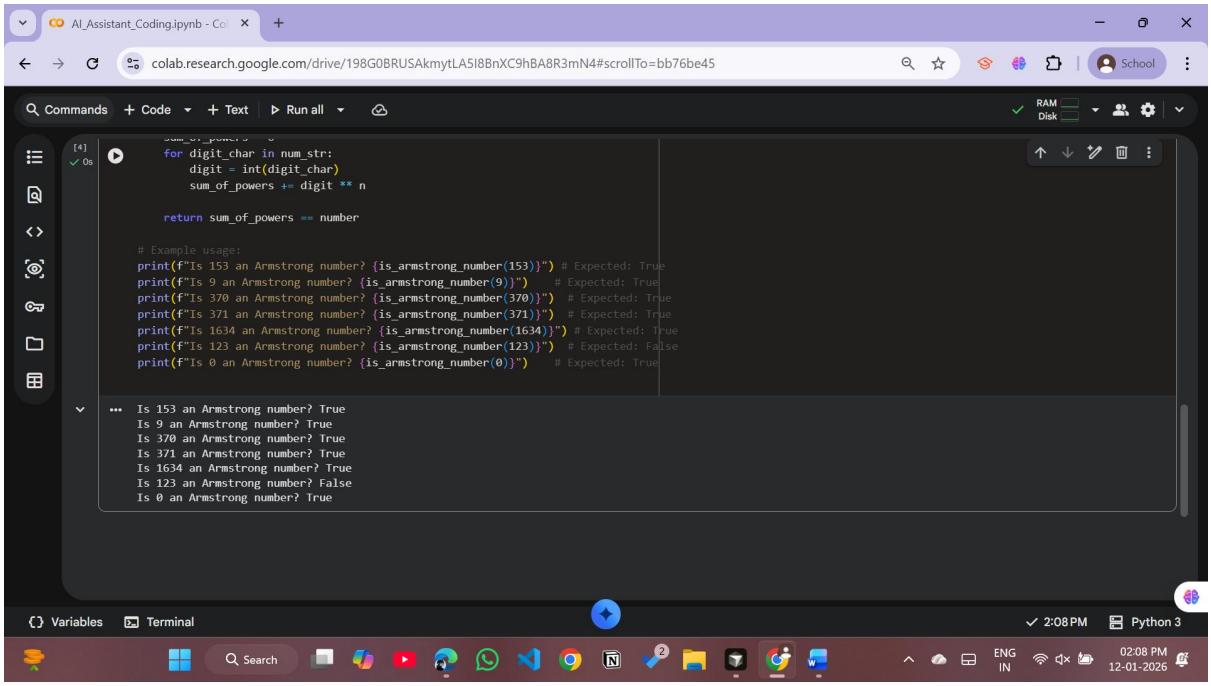
Side-by-side comparison table

Screenshots of prompts and generated code.

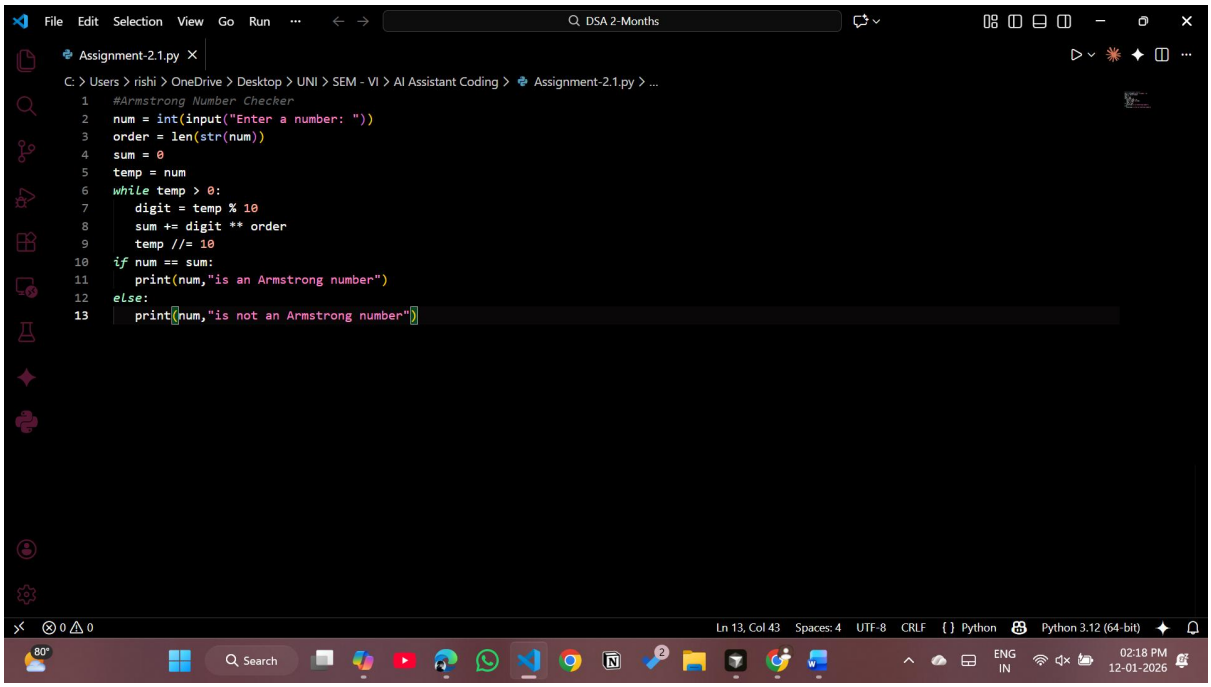
Gemini (GoogleColab):

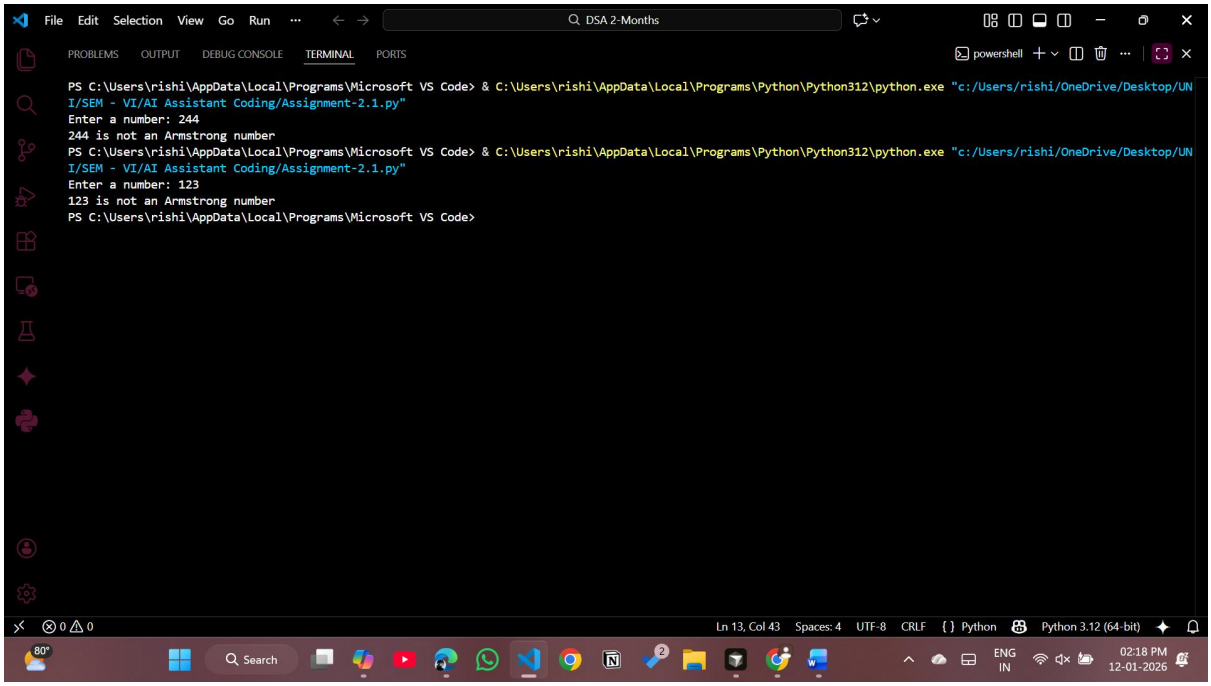






GithubCopilot (VSCode):





Improvements after comparing Gemini and GitHub Copilot outputs:

- The AI-generated code improved **logical clarity**, making the Armstrong calculation easier to understand.
- Variable naming became more meaningful, which improved **code readability**.
- The refactored version reduced unnecessary steps, making the program **more efficient**.
- Use of loops and arithmetic operations was optimized for better **performance**.
- Overall structure became more **beginner-friendly**, helping in learning numeric validation logic.

Task 3: Leap Year Validation Using Cursor AI

Scenario: You are validating a calendar module for a backend system.

Task: Use Cursor AI to generate a Python program that checks whether a given year is a leap year. Use at least two different prompts and observe changes in code.

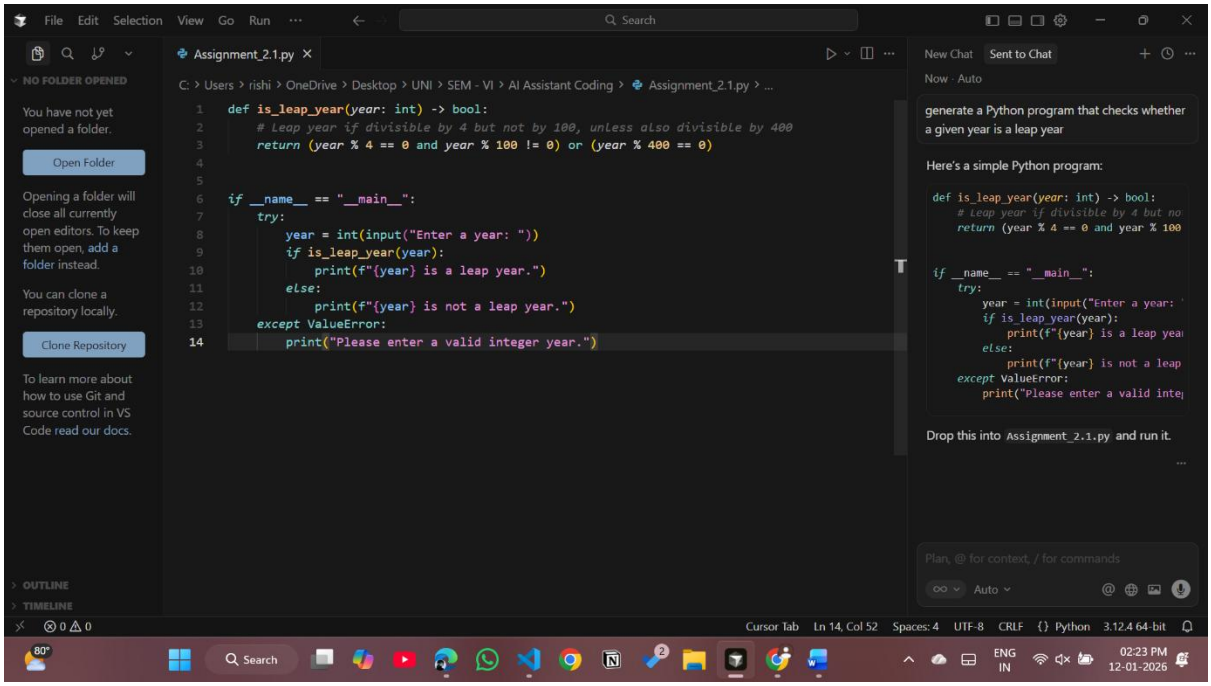
Expected Output:

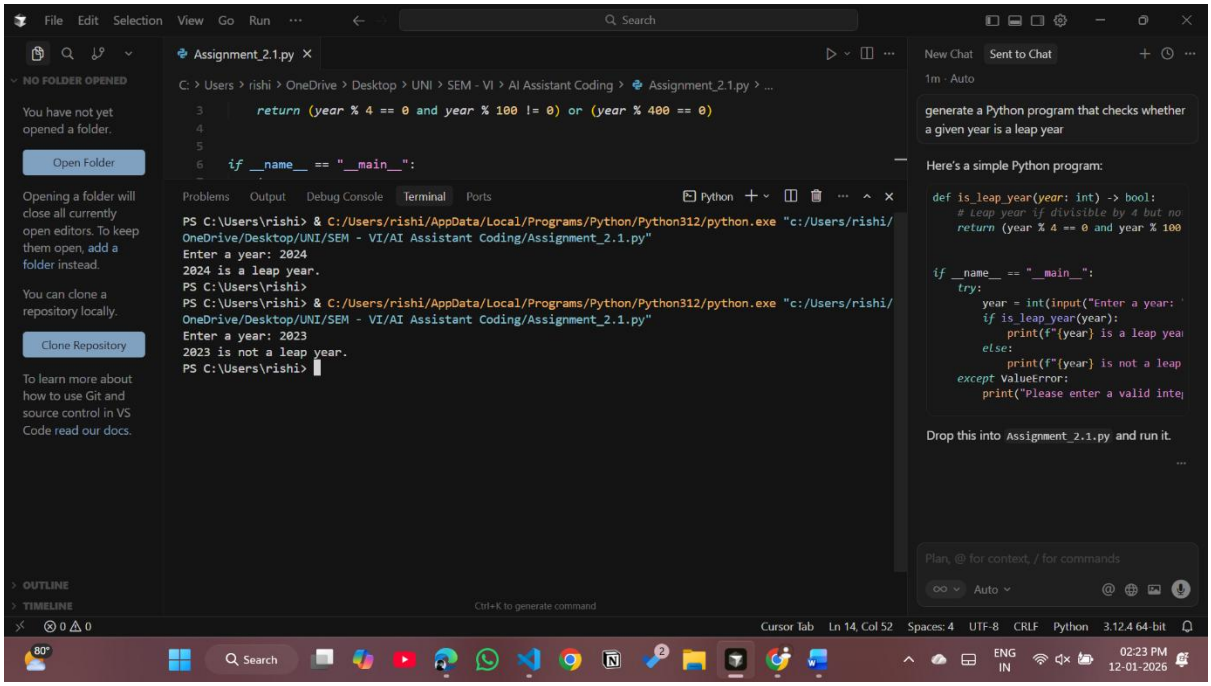
Two versions of code

Sample inputs/outputs

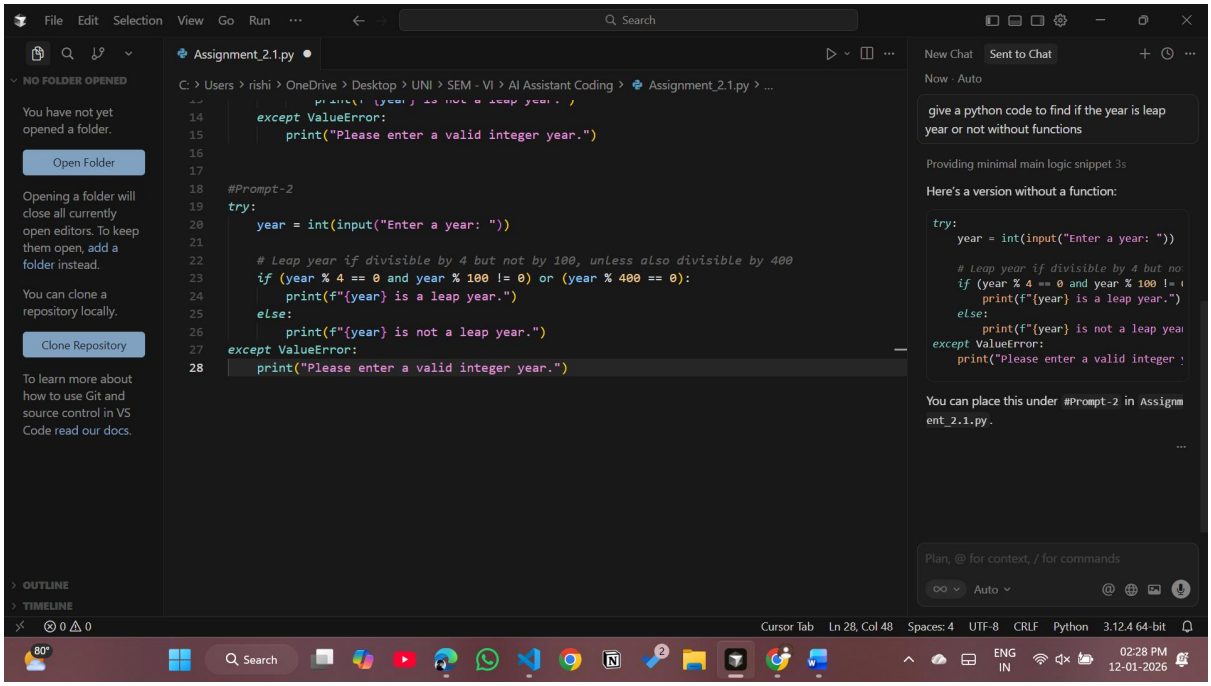
Brief comparison

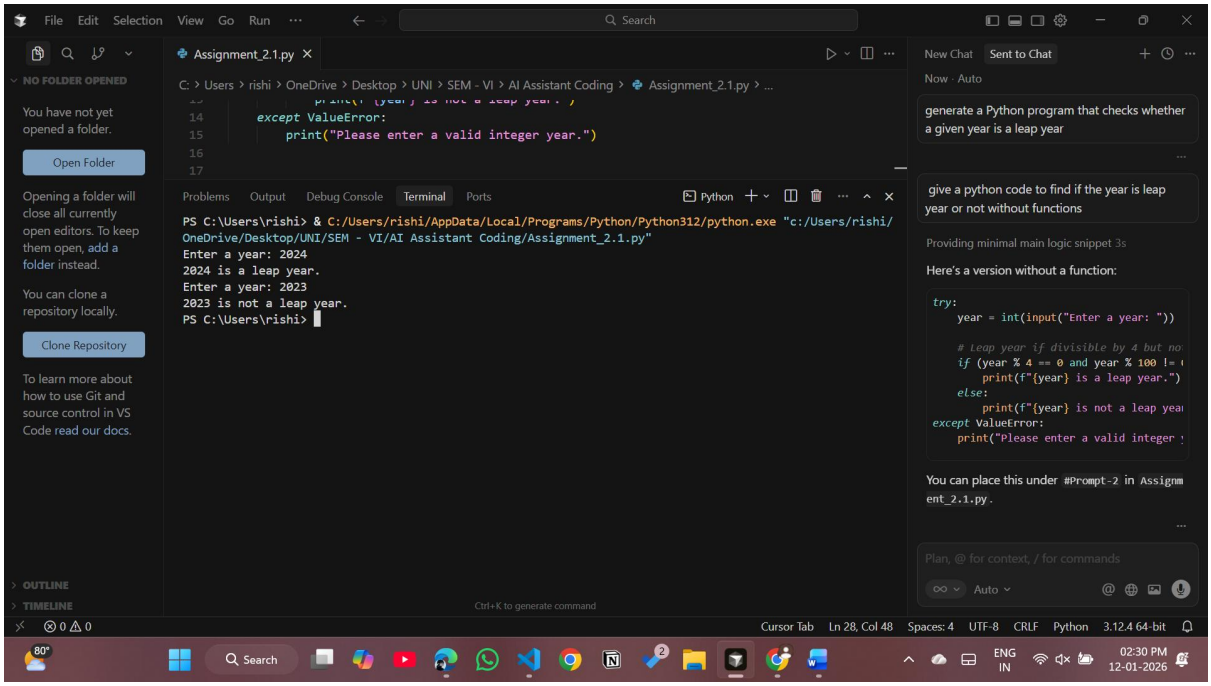
Prompt-1: generate a Python program that checks whether a given year is a leap year.





Prompt-2: give a python code to find if the year is leap year or not without functions.





Improvements observed across different prompts:

- Changing the prompt resulted in **clearer logical flow** in the second version of the code.
- Step-by-step conditional checks improved **readability and understanding**.
- The refined code avoided ambiguity in leap year rules, ensuring **logical correctness**.
- Prompt variation demonstrated how AI can adapt logic based on **developer intent**.
- The final code became easier to test and debug due to **explicit conditions**.

Task 4: Student Logic + AI Refactoring (Odd/Even Sum)

Scenario: Company policy requires developers to write logic before using AI.

Task: Write a Python program that calculates the sum of odd and even numbers in a tuple, then refactor it using any AI tool.

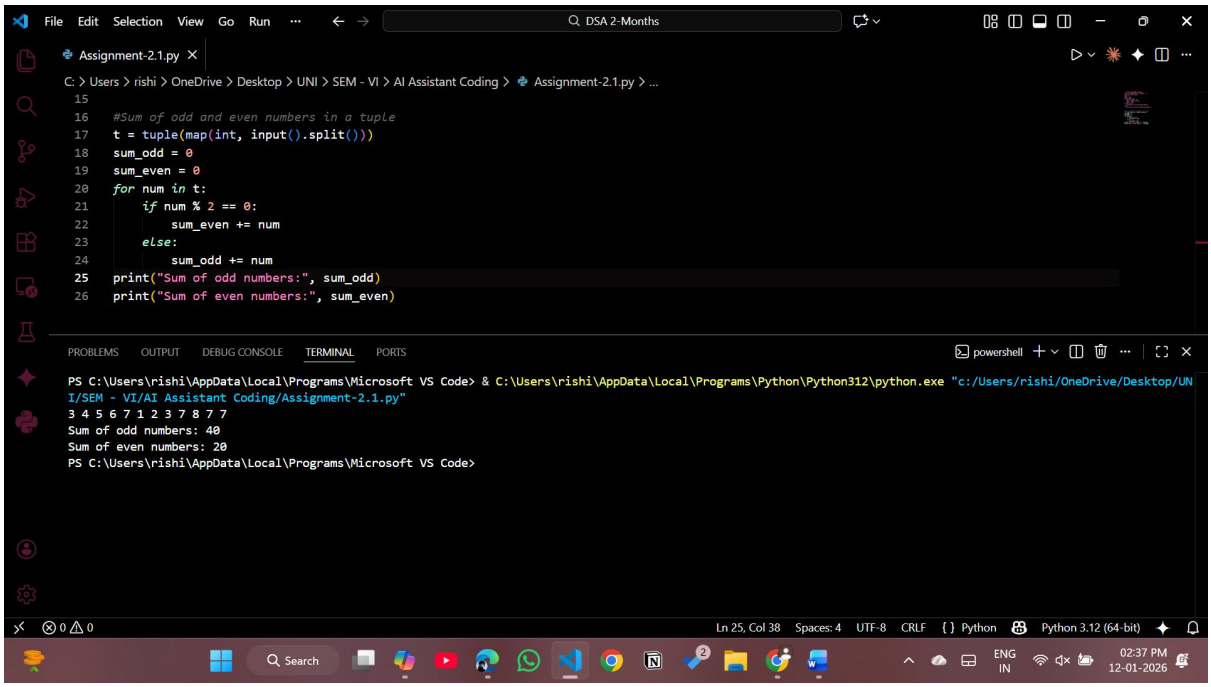
Expected Output:

Original code

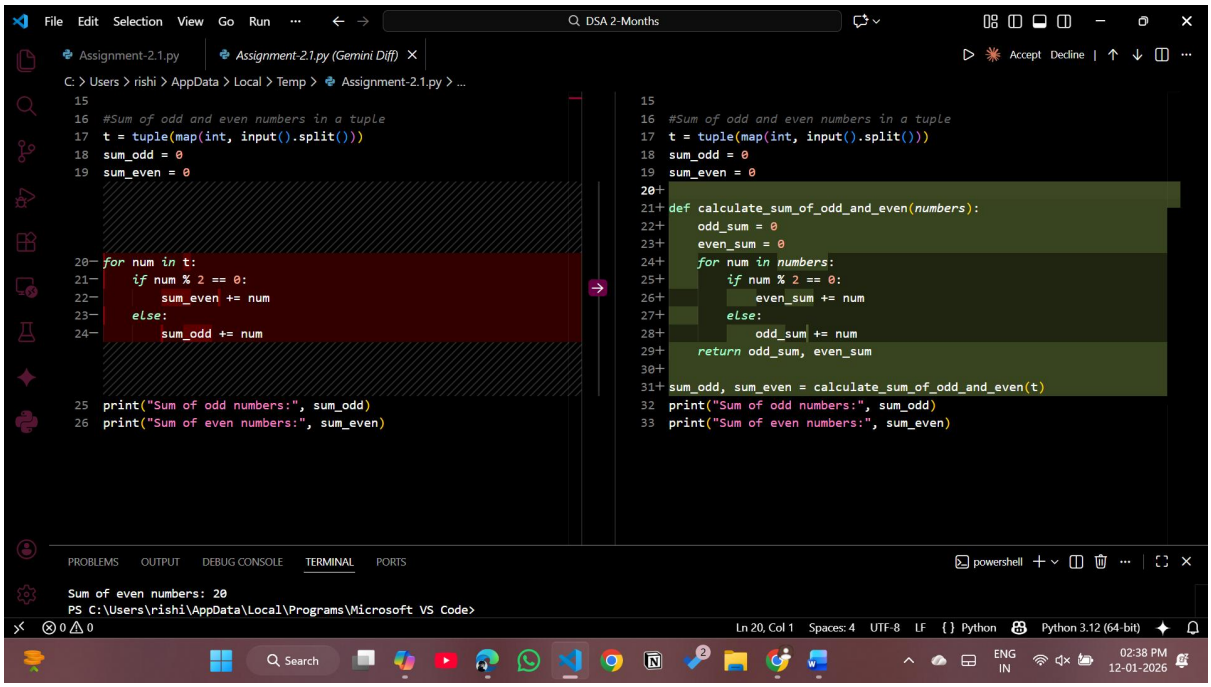
Refactored code

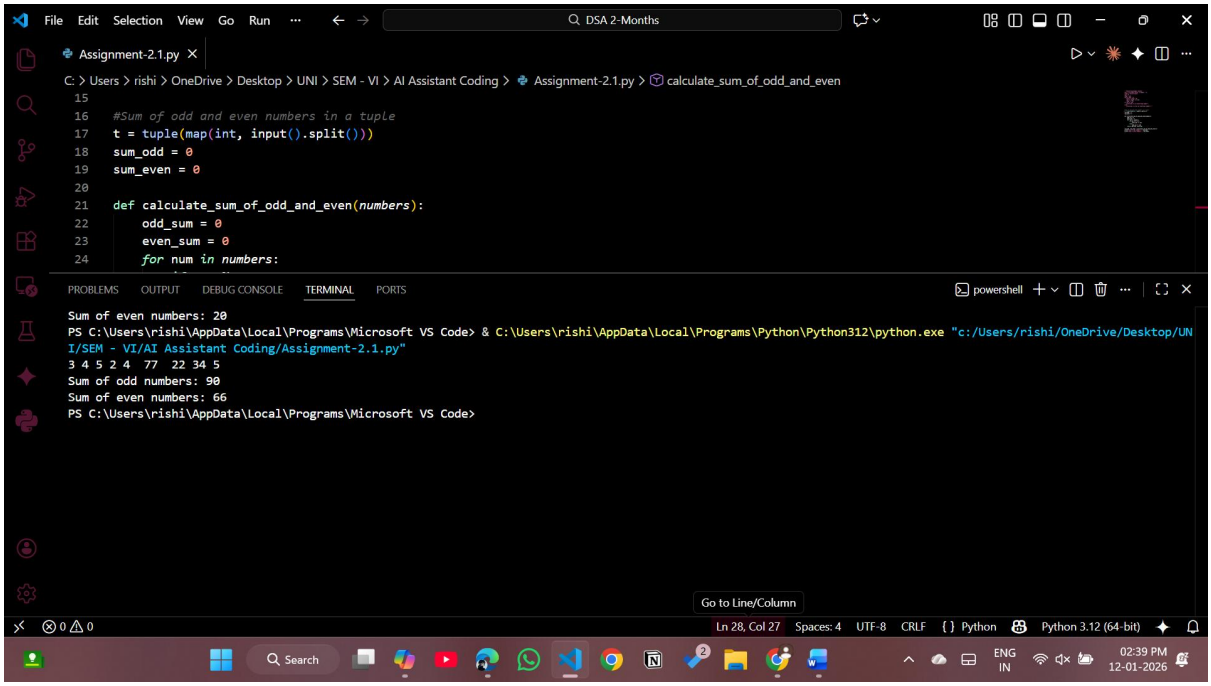
Explanation of improvements

Original Code:



Refactored Code:





Improvements after AI-based refactoring:

- Logic was modularized using a function, improving **code reusability**.
- The refactored code follows **better programming practices**.
- Maintenance became easier since changes are limited to a single function.
- Improved structure enhanced **readability and scalability**.
- Demonstrates how AI helps refine existing logic without changing functionality.